

Concurrency and Distributed Systems

Week 7 – Part 2: Timeout, Fallback, and Failure

Dr. Mohamad Aoude

Faculty of Engineering

May 13, 2026

Part 2 Roadmap

1 Timeouts

2 Failures

3 Anti-Patterns

4 Demo

Why Timeout Exists

- A slow dependency can hold a caller forever.
- Async code without timeout still has a liveness problem.
- A timeout turns unbounded waiting into a bounded design decision.
- Timeout values are part of the service contract.

Fallback With completeOnTimeout

```
1 CompletableFuture<String> result =  
2     slowService()  
3     .completeOnTimeout(  
4         "fallback-profile",  
5         100,  
6         TimeUnit.MILLISECONDS);
```

Meaning

If the service completes in time, use the real value. If it does not, complete with a known fallback value.

Exception Recovery

```
1 CompletableFuture<ServiceResult> recommendations =  
2     callRecommendations()  
3     .exceptionally(ex ->  
4         new ServiceResult("recommendations",  
5                             "fallback",  
6                             true));
```

- Recover close to the failed branch.
- Mark that fallback happened.
- Do not convert every failure into a vague default silently.

Failure Handling Choices

Method	Use	Risk
<code>exceptionally</code>	map failure to fallback	can hide cause
<code>handle</code>	inspect success or failure	can become messy
<code>whenComplete</code>	observe outcome	does not recover alone
<code>orTimeout</code>	fail after deadline	caller must handle failure
<code>completeOnTimeout</code>	fallback after deadline	fallback must be visible

Async Anti-Patterns

Thread per request

Easy to write, but it scales poorly when many calls block.

Missing timeout

A future that never completes is still a stuck workflow.

Silent fallback

If the response hides that fallback happened, observability and correctness suffer.

Run

```
.\run-demo.ps1 02
```

- First run: service completes before timeout.
- Second run: timeout returns fallback.
- Ask students where the caller learns fallback happened.

Key Takeaway

Async reliability needs explicit policy

Define when to wait, when to stop waiting, what fallback means, and how fallback is reported.